Drift-Aware Memory-Bounded Batching for Irregular GPU Pipelines

An extension study of explicit resource bounds for out-of-kernel GPU optimization

Lee Si Yuan Independent Researcher siyuanlee.cs@gmail.com

Abstract

Memory-bounded batching removes reliance on backend occupancy estimates by filling a worker's memory budget with as much work as its resource model permits. That policy assumes the estimated per-item footprint remains representative as an input stream evolves. We propose **Drift-Aware Memory-Bounded Batching (DAMB)**, a runtime controller that tracks per-class footprint estimates and uses a conservative, smoothed estimate to revise safe batch capacities when workload characteristics change. DAMB keeps explicit memory bounds as a hard constraint and leaves kernel code unchanged. We implement a small deterministic CPU simulation of a manager/worker pipeline with three workload phases and compare first-phase static calibration against an online smoothed estimate with a fixed safety reserve. In this deliberately simplified model, the online policy reaches 1.822 normalized items per time unit versus 1.805 for the static policy (1.009×); the small difference is sensitive to integer batch partitioning and is not evidence of a GPU speedup. The experiment verifies the controller's execution path and exposes a central limitation: when classes have stable, known footprints, adapting estimates adds little value. We specify a stronger drift-triggered design and an evaluation protocol for real GPU systems. Results are simulation-only; no docking code, molecular data, or GPU hardware was used.

Keywords: GPU batching, performance portability, workload drift, memory bounds, manager/worker execution

1. Introduction

Irregular GPU pipelines often spend significant time in host-side decisions: grouping inputs, allocating buffers, and submitting work across workers and streams. Accordi et al. study these out-of-kernel decisions in the muDock molecular docking mini-app. Their memory-bounded policy chooses the largest class-specific batch that fits the memory assigned to a worker, avoiding occupancy hints that may differ across backends [1].

A fixed footprint model is useful when a run has a stable input distribution. Screening workloads can instead change over time: a queue may move from mostly small ligands to a more flexible subset, or an inference service may receive a new request mix. A stale estimate can either leave memory unused or lead to an unsafe allocation. The extension studied here is a bounded feedback loop that observes completed batches and updates the next batch capacity.

This paper contributes (1) a simple online estimate-and-reserve rule, (2) a deterministic CPU simulation that makes its assumptions explicit, and (3) a proposed drift-triggered controller and GPU evaluation protocol. The experiment is a feasibility check, not a reproduction of the hardware results in [1].

2. Background and motivation

The base study compares occupancy-guided batching with memory-bounded batching in a manager/worker execution model. Each worker submits batches on an independent stream. The base memory bound is

> B_max(c) = floor(M_worker / m_hat(c)),

where M_worker is the worker's memory budget and m_hat(c) is the estimated per-item footprint for input class c. Occupancy-guided batching additionally aligns batch sizes with backend-specific active-block estimates. The reported cross-platform study finds that memory-bounded configurations match or exceed the occupancy-guided peak throughput across the tested platform/backend pairs, while the fully tuned configuration averages 1.53× over its reference setup [1].

DAMB preserves this organization and changes how m_hat(c) is maintained. The key question is conditional: does online tracking help when class composition or within-class footprints drift, while preserving the resource-safety property?

3. Method: Drift-Aware Memory-Bounded Batching

3.1 Online estimate

For each class c, maintain an exponentially smoothed footprint estimate:

> m_hat_t(c) = alpha * m_obs_t(c) + (1 - alpha) * m_hat_(t-1)(c),

where m_obs_t(c) is measured from the latest completed batch and alpha is a smoothing parameter. The initial estimate can come from a short warm-up sample or a declared conservative bound.

3.2 Conservative batch capacity

Choose a reserve factor rho in (0, 1] and compute

> B_t(c) = max(1, floor(rho * M_worker / m_hat_t(c))).

The reserve factor accounts for model error and non-payload allocations. It must be calibrated with measured peak allocation, not selected from throughput alone. Implementations with a reliable hard upper bound m_max(c) should enforce the stronger safety check

> B_t(c) * m_max(c) + M_overhead <= M_worker.

The smoothed estimate affects efficiency; the hard bound protects correctness. An estimate alone cannot guarantee safety under an unbounded distribution shift.

3.3 Drift trigger and controller policy

The prototype updates estimates after each phase. A production controller should update after every fixed observation window and change a capacity only when the relative estimate shift exceeds a threshold delta:

> |m_obs(c) - m_hat(c)| / max(m_hat(c), epsilon) > delta.

On a trigger, it should lower the next batch capacity immediately, then increase capacity gradually after sustained lower observations. This asymmetric response limits overshoot after a sudden increase in footprint. A production implementation should also cap the estimate using a known class bound or sample a high percentile with a conservative fallback. If no safe bound is available, it should retain the existing safe capacity until enough observations are collected.

The controller does not alter kernel implementations or require occupancy APIs. Its state is per class and per device; separate state avoids transferring one backend's calibration to another backend.

4. Experimental setup

We implemented `adaptive_batching_experiment.py`, a deterministic Python simulation using seed 20261106. Each of three phases contains 2,400 items, with light-item proportions 0.90, 0.25, and 0.90. Light items use one normalized memory unit and one work unit; heavy items use eight memory units and six work units. The worker budget is 96 units, four workers execute batches, and each batch adds two normalized launch-cost units.

The static baseline uses the initial class footprint estimates and the full memory budget. The online policy uses the same starting estimates, an 0.85 reserve factor, and 0.5 exponential smoothing after each phase. Class-specific jobs are grouped, then batches are greedily assigned to the least-loaded worker. The simulated time is the maximum worker load for each phase; phase times are summed. These choices produce a transparent scheduling sensitivity model, not a calibrated model of CUDA, HIP, SYCL, muDock, or any GPU.

The run can be reproduced from the repository root with:

```
python papers/feedback_batching/adaptive_batching_experiment.py
```

It writes `results.json` beside the script.

5. Results

Policy	Simulated total time	Aggregate throughput (items / normalized time)	Relative throughput
Static first-phase calibration	3988	1.805	1.000×
Online smoothed estimate, 15% reserve	3952	1.822	1.009×

The online policy differs by less than 1% in this setup. Its phase throughput is 4.131, 0.860, and 4.131, compared with 4.082, 0.853, and 4.082 for static calibration. The controller increases batch count because of its reserve; integer batch partitioning and greedy worker assignment determine the small timing difference. Since item footprints are constant within each class and the class estimates already equal those values, this workload does not meaningfully exercise estimation drift. It is therefore a negative result for claims of general benefit: the prototype does not establish that online adaptation improves a real workload.

6. Discussion and limitations

The experiment confirms that the code path is reproducible and that changing batch capacity changes the worker schedule. It also demonstrates why a tiny positive result in a synthetic model must not be presented as a GPU performance claim. The input has only two fixed classes, no transfers, no allocation fragmentation, no kernel execution, no concurrent external GPU users, and no measurement noise. The 0.9% difference can arise from discrete batch scheduling rather than useful adaptation.

The proposed method is most plausible when per-class footprint estimates drift over time and the runtime can observe allocations or obtain safe class bounds. It may be ineffective when the base class model is accurate, as in this simulation. It can reduce throughput if the reserve is too conservative, and an average estimate alone can be unsafe for heavy-tailed items. The hard-bound rule and an asymmetric update are necessary for deployment.

A real evaluation should integrate DAMB into muDock or a comparable pipeline, replay time-ordered workloads with distribution shifts, and compare (a) fixed memory-bounded batching, (b) online mean estimates, and (c) high-percentile estimates with a hard safety bound. Report throughput, peak allocated memory, allocation failures, batch wait time, worker idle time, and per-phase results across repeated runs on each supported backend. Ablations should vary alpha, delta, reserve factor, and drift magnitude. No such GPU evaluation is claimed here.

7. Conclusion

We proposed a small feedback extension to memory-bounded batching: track per-class footprints, preserve a conservative resource bound, and revise future batch capacities when measurements indicate drift. The executed CPU simulation shows near parity under stable per-class footprints, which is consistent with adaptation having little to learn in that case. The main research question remains open: whether a safe controller can improve throughput under realistic within-class drift without increasing memory failures. The supplied experiment and results are intended as a reproducible starting point for that hardware study.

References

[1] G. Accordi, L. Beltrame, T. Pollinger, J. Domke, D. Gadioli, and G. Palermo, “Performance Portability of Out-of-Kernel Optimizations on GPUs: A Molecular Docking Case Study,” arXiv:2611.06207v1, 2026. https://arxiv.org/abs/2611.06207.

Author information

Lee Si Yuan · Independent Researcher · siyuanlee.cs@gmail.com